

Solving flexible job shop scheduling problems via deep reinforcement learning

Erdong Yuan^a, Liejun Wang^{a,*}, Shuli Cheng^a, Shiji Song^b, Wei Fan^c, Yongming Li^a

^a College of Information Science and Engineering, Xinjiang University, Urumqi, 830017, Xinjiang, China

^b Department of Automation, Tsinghua University, Beijing, 100084, Beijing, China

^c AI Laboratory, Lenovo Research, Beijing, 100085, Beijing, China

ARTICLE INFO

Keywords:

Flexible job shop scheduling problem
Deep reinforcement learning
Multi-layer perceptron
State representation
Generalization

ABSTRACT

Flexible job shop scheduling problem (FJSSP), as a variant of the job shop scheduling problem, has a larger solution space. Researchers are always looking for good methods to solve this problem. In recent years, the deep reinforcement learning (DRL) has been applied to solve various shop scheduling problems due to its advantages that fast solving speed and strong generalization ability. In this paper, we first propose a new DRL framework to realize representation learning and policy learning. The new framework adopts a lightweight multi-layer perceptron (MLP) as the state embedding network to extract state information, which reduces the computational complexity of the algorithm to some extent. Next, we design a new state representation and define a new action space. The new state representation can directly reflect the state features of candidate actions, which is conducive for the agent to capture more effective state information and improve its decision-making ability. The new definition of action space can solve the two subproblems of the FJSSP simultaneously with only one action space. Finally, we evaluate the performance of the policy model on four public datasets: Barnes dataset, Brandimarte dataset, Dauzere dataset and Hurink dataset. Extensive experimental results on these public datasets show that the proposed method achieves a better compromise in terms of optimization ability and applicability compared to the composite priority dispatching rules and the existing state-of-the-art models.

1. Introduction

Flexible job shop scheduling problem (FJSSP) is one of NP-Hard combinatorial optimization problems (Sotskov & Shakhlevich, 1995; Xie, Gao, Peng, Li, & Li, 2019). Compared with the job shop scheduling problem (JSSP), solving FJSSP is equivalent to solving two subproblems, namely the operation sequencing subproblem and the machine allocation subproblem. Current methods for solving FJSSP are mainly divided into two types: exact methods and approximate methods. Exact methods have strong optimization ability, but they have high computational complexity, which are only suitable to solve some small-scale problems, such as mixed integer linear programming (Meng, Zhang, Shao, & Ren, 2019). Approximate methods mainly include heuristic methods, meta-heuristic methods and learning-based methods. They can obtain satisfactory solutions in a relatively short time. Heuristic methods are mainly some priority dispatching rules (PDRs), which have the advantage of fast computing speed. However, they are often short-sighted because they fail to fully consider the properties and characteristics of the problem to be solved. At the same time, no priority

dispatching rule (PDR) is suitable to deal with all problems, as proved in the *No Free Lunch Theorem For Optimization* (Wolpert & Macready, 1997). Meta-heuristic methods are mainly some iterative search algorithms that are inspired from some natural phenomena. Given some initial solutions, they improve the quality of the solutions by continuous iterative search. However, the design of the meta-heuristic method is complicated and the stability of the solution quality cannot be guaranteed due to the random factors in the algorithm. Learning-based methods can be divided into supervised learning and reinforcement learning (RL) in terms of learning paradigm. The former needs to use the optimal solution or approximate optimal solution of the problem as data labels of the training dataset. However, it is often computationally complex for obtaining data labels. In contrast, the latter is not limited by data labels. Meanwhile, deep reinforcement Learning (DRL) has the advantages of fast solving speed and strong generalization ability, so it has become a research hotspot for solving FJSSP.

Some current DRL methods still have deficiencies in the state representation, the definition of action space and the design of reward

* Corresponding author.

E-mail addresses: yerdong@stu.xju.edu.cn (E. Yuan), wljxju@xju.edu.cn (L. Wang), cslxju@xju.edu.cn (S. Cheng), shijis@mail.tsinghua.edu.cn (S. Song), fanwei2@lenovo.com (W. Fan), lym@xju.edu.cn (Y. Li).

<https://doi.org/10.1016/j.eswa.2023.123019>

Received 21 February 2023; Received in revised form 21 December 2023; Accepted 21 December 2023

Available online 29 December 2023

0957-4174/© 2023 Elsevier Ltd. All rights reserved.

function. Some methods make it difficult for the agent to capture effective state information in the state representation, to explore solution space effectively in the definition of action space, and to quickly identify key action in the design of reward function due to sparse reward. Besides, the performance of the policy model is also affected by the different state embedding networks, which is reflected in the calculation complexity of the algorithm to some extent. In this paper, we mainly focus on the state representation and the definition of action space in Markov decision process (MDP) modeling. Besides, we use a lightweight network model to extract state information.

The main contributions in this paper can be summarized as follows:

- We propose a new DRL framework to realize representation learning and policy learning. Unlike other learning-based methods that use the node message aggregation of a complex graph neural network (GNN) to extract the state information, the new DRL framework adopts a lightweight multi-layer perceptron (MLP) as the state embedding network, which reduces the computational complexity of the algorithm to some extent.
- We design a new state representation. The new state representation can directly reflect the state features of candidate actions, which is conducive for the agent to capture more effective state information and improve its decision-making ability.
- We define a new action space. Unlike other learning-based methods that typically define two action spaces to solve two subproblems of FJSSP, the new action space definition can solve both subproblems of FJSSP simultaneously with only one action space, meaning that only one policy model is trained.
- To verify the effectiveness of the proposed method, we evaluate the performance of the policy model on four public datasets: Barnes dataset (Barnes & Chambers, 1996), Brandimarte dataset (Brandimarte, 1993), Dauzere dataset (Dauzère-Pérès & Paulli, 1997) and Hurink dataset (Hurink, Jurisch, & Thole, 1994). Compared with the current state-of-the-art models and well-known composite PDRs, extensive experimental results show that the proposed method has superior performance on four public datasets.

The rest of the content is organized as follows: Section 2 reviews related work, which mainly involves the research status of approximation methods for solving the shop scheduling problem; Section 3 describes the problem; The implementation details of the proposed method are given in Section 4; Section 5 presents the experimental results and analysis; Conclusions are given in Section 6.

2. Related work

Considering that exact algorithms are suitable to solve some small-scale problems, this section mainly reviews some current approximation methods.

2.1. Heuristic and meta-heuristic methods

There is an essential difference between the heuristic method and the meta-heuristic method in terms of operating mechanism: the heuristic method is a constructive method starting from no scheduling; the meta-heuristic method is an improved method, that is, on the basis of the existing complete scheduling schemes, the quality of the solution is further improved by continuous iterative search. Heuristic methods for solving FJSSP are mainly some composite PDRs (Sobeyko & Mönch, 2016; Thenarasu, Rameshkumar, Rousseau, and Anbuudayasankar, 2022), that is, the combination of operation sequencing rules and machine allocation rules. The operation sequence rules (Chiang & Fu, 2007; Haupt, 1989; Narassima, Vasudevan, & Anbuudayasankar, 2022) can also be used to solve JSSP, such as *First In First Out* (FIFO), *Most Work Remaining* (MWKR), and *Most Operation Number Remaining* (MOPNR). The machine allocation rules (Doh, Yu, Kim, Lee, & Nam,

2013) include *Shortest Processing Time* (SPT), *Earliest End Time* (EET), etc. Meta-heuristic methods for solving FJSSP are mainly some iterative search algorithms, such as tabu search algorithm (Saidi-Mehrabad & Fattahi, 2007), artificial immune algorithm (Bagheri, Zandieh, Mahdavi, & Yazdani, 2010), ant colony optimization algorithm (Xing, Chen, Wang, Zhao, & Xiong, 2010), jaya algorithm (Caldeira & Gnanavelbabu, 2019), and genetic algorithm (Chen, Yang, Li, & Wang, 2020; Pezzella, Morganti, & Ciaschetti, 2008; Rooyani & Defersha, 2019). In addition, there are also some hybrid algorithms (Gao et al., 2019; Karthikeyan, Asokan, Nickolas, & Page, 2015; Tang, Zhang, Lin, & Zhang, 2011; Yuan, Xu, & Yang, 2013).

2.2. Learning-based methods

The successful application of the DRL method in the field of Go game, such as Atari (Mnih et al., 2013), AlphaGo (Silver et al., 2017), and AlphaStar (Vinyals et al., 2019), has greatly encouraged researchers who hope to apply this emerging technology to solve various shop scheduling problems (Brammer, Lutz, & Neumann, 2022; Han, Guo, & Su, 2019; Yan, Wu, & Wang, 2022). Researchers focused on solving JSSP via DRL method in the early stage, and then transitioned to solving FJSSP. For JSSP, only the operation sequencing problem needs to be considered. However, for FJSSP, we need to consider not only the operation sequencing problem, but also the machine allocation problem. It is worth affirming that some works solving JSSP via DRL have good guiding significance for applying this method to solve FJSSP. Table 1 summarizes some recent works for solving JSSP and FJSSP via DRL.

2.2.1. Solving JSSP via DRL

Lin et al. (2019) combined deep Q-network (DQN) and edge computing to solve JSSP. This problem is a kind of JSSP for the semiconductor industry and has specific application scenarios. Han and Yang (2020) trained the policy model by Dueling Double DQN to solve adaptive JSSP. However, the training dataset and the test dataset adopted by the authors are the same, which may lead to overfitting. Zhang et al. (2020) combined the graph isomorphic network (GIN) and the proximal policy optimization (PPO) algorithm to solve JSSP. The state information is extracted through the node message aggregation of GIN, and the generalization of the policy model is used to solve other scale problems, especially large-scale problems. However, this method has insufficient in the state representation and the definition of action space, which make it difficult for the agent to learn more effective policy. Wang et al. (2021) adopted the DRL method to solve dynamic small-scale JSSP. However, the policy model trained cannot guarantee the stability of the model performance during the test phase, which may be due to the existence of multiple optimal action selections among the candidate actions. Park et al. (2021) took the GNN as the state embedding network, and the state features contain six different types of inputs to represent various relationships between nodes. Meanwhile, the PPO algorithm is used to update the network model parameters. Yuan et al. (2023) solved JSSP via DRL. The state features of bidirectional scheduling were used as state representation to make the agent capture more effective state information, and the invalid action masking technique was adopted to narrow the search space and avoid exploring some suboptimal solutions.

2.2.2. Solving FJSSP via DRL

Park et al. (2019) solved FJSSP for the semiconductor industry on multi-chip products (MCPs) via Q-network. Luo (2020) trained the policy model by DQN to solve dynamic FJSSP with new job insertions. Luo et al. (2021) designed a two-layer DQN framework to solve dynamic multi-objective FJSSP. Its action space is composed of six composite PDRs. However, due to the short-sightedness of PDRs, local optimal solutions cannot be guaranteed. Du et al. (2022) combined DQN with estimation of distribution algorithm (EDA) to solve FJSSP with time-of-use

Table 1

Survey of the learning-based method for solving shop scheduling problems.

Methods	Algorithm	Representation learning	Agent	Objective	Dynamic event	Problem type
Lin, Deng, Chih, and Chiu (2019)	Value-Based	DNN	Single-agent	Makespan	No	JSSP
Han and Yang (2020)	Value-Based	CNN	Single-agent	Makespan	No	JSSP
Zhang et al. (2020)	Policy-Based	GIN	Single-agent	Makespan	No	JSSP
Wang et al. (2021)	Policy-Based	DNN	Single-agent	Makespan	Yes	JSSP
Park, Chun, Kim, Kim, and Park (2021)	Policy-Based	GNN	Single-agent	Makespan	No	JSSP
Yuan, Cheng, Wang, Song, and Wu (2023)	Policy-Based	GNN	Single-agent	Makespan	No	JSSP
Park, Huh, Kim, and Park (2019)	Value-Based	DNN	Multi-agent	Makespan	No	FJSSP
Luo (2020)	Value-Based	DNN	Single-agent	Total tradiness	Yes	FJSSP
Luo, Zhang, and Fan (2021)	Value-Based	DNN	Single-agent	Multi-objective	Yes	FJSSP
Du, Li, Chen, Duan, and Pan (2022)	Value-Based	FCNN	Multi-agent	Multi-objective	No	FJSSP
Song, Chen, Li, and Cao (2022)	Policy-Based	GNN	Multi-agent	Makespan	No	FJSSP
Lei et al. (2022)	Policy-Based	GNN	Single-agent	Makespan	No	FJSSP
Zhang, He, Chan, and Chow (2023)	Value-Based	DNN	Multi-agent	Makespan	No	FJSSP
Ours	Policy-Based	DNN	Single-agent	Makespan	No	FJSSP

electricity price constraints, whose DQN topology is a fully connected neural network (FCNN), and the optimization objectives are to minimize makespan and total electricity price. Song et al. (2022) adopted a heterogeneous GNN to embedding operation node features, machine node features, and O-M arc features, and used the PPO algorithm to train the policy model on synthetic instances of four small and medium-scale problems to solve FJSSP. During the test phase, the authors adopted a greedy strategy and a sampling strategy to select actions, respectively. It is worth noting that although the sampling strategy can effectively improve the solution quality, it increases the computational complexity. Lei et al. (2022) proposed a multi-action DRL framework to learn two sub-policy models for solving FJSSP, and the two sub-policy models are used to select the operation node and the machine node at each decision-making step. The authors adopt GIN and a fully connected layer to realize the state features embedding of operation node and machine node, respectively. The policy model is trained using the PPO algorithm based on small and medium-sized problems. Zhang et al. (2023) combined DQN with multi-agent graphs to solve FJSSP, and evaluated the performance of the model in three scenarios (with 20, 60, and 100 jobs, respectively), where the multi-agent graphs represent the operational relationships between machines and jobs.

At present, research into DRL methods for solving JSSP or FJSSP is still in its infancy. Different DRL methods have great differences in optimization objective settings, training algorithms, test datasets, and even some methods are specific to FJSSP with specific application scenarios. These differences make it difficult for comparison between different DRL methods. To facilitate comparison with other DRL methods, we take minimizing the makespan as the optimization objective in this paper and evaluate the model performance on four public datasets.

3. Problem description

The standard FJSSP is described as follows: There are n jobs $J = \{J_1, J_2, \dots, J_i, \dots, J_n\}$ that need to be processed on m machines $M = \{M_1, M_2, \dots, M_k, \dots, M_m\}$. Each job contains at least one operation. For an operation, only one machine can be selected from the optional machine set to process the operation, and the processing time of an operation is different on optional machines. The optimization objective of FJSSP is to minimize the makespan under the premise of satisfying various constraints. Priority constraints need to be satisfied between adjacent operations of the same job, and capacity constraints also need to be met, that is, the same machine can only process at most one operation at a time. Completing the scheduling task of FJSSP is equivalent to solving two subproblems, namely the job sequencing subproblem and the machine allocation subproblem. Some assumptions that need to be satisfied to solve FJSSP are as follows:

- (1) The transport time between operations can be ignored or included in the processing time;
- (2) Each operation is not allowed to be interrupted during processing.

In order to express FJSSP more intuitively, some symbols of FJSSP are defined as follows:

(1) Parameters:

- n : total number of jobs;
- m : total number of machines;
- i, h : index of jobs, $i, h = 1, \dots, n$;
- n_i : the number of operations of job i ;
- n_{max} : the maximum number of operations for all jobs;
- j, g : index of operations, $j, g = 1, \dots, n_i$;
- k, e : index of machines, $k, e = 1, \dots, m$;
- O_{ij} : the j th operation of job i ;
- Ω_{ij} : the optional machine set of O_{ij} ;
- O_{ijk} : the j th operation of job i is processed on machine M_k ;
- p_{ijk} : the processing time of O_{ij} on machine M_k ;
- $\bar{p}_{ij}$: the average processing time of O_{ij} on Ω_{ij} ;
- s_{ij} : the start time of O_{ij} ;
- c_{ij} : the completion time of O_{ij} ;
- L : a sufficiently large positive number;
- C_i : the completion time of job i ;
- C_{max} : makespan.

(2) Decision variables:

$$x_{ijk} = \begin{cases} 1, & \text{if } O_{ij} \text{ selects processing machine } M_k; \\ 0, & \text{otherwise;} \end{cases}$$

$$y_{ijhgk} = \begin{cases} 1, & \text{if } O_{ijk} \text{ is a predecessor of } O_{hgk}; \\ 0, & \text{otherwise;} \end{cases}$$

Combined with the above symbol definitions and the model developed by Fattahi, Saidi Mehrabad, and Jolai (2007), the mathematical model of FJSSP can be expressed as follows:

Minimize:

$$C_{max} = \max_{1 \leq i \leq n} \{C_i\} \quad (1)$$

Subject to:

$$s_{ij} + x_{ijk} \times p_{ijk} \leq c_{ij} \quad (2)$$

$$c_{ij} \leq s_{i(j+1)} \quad (3)$$

$$s_{ij} + p_{ijk} \leq s_{hg} + L(1 - y_{ijhgk}) \quad (4)$$

$$c_{ij} \leq s_{i(j+1)} + L(1 - y_{hgi(j+1)k}) \quad (5)$$

$$\left| \Omega_{ij} \right| \sum_{k=1} x_{ijk} = 1 \quad (6)$$

Eq. (1) indicates that the optimization objective is to minimize the makespan C_{max} ; Eqs. (2) and (3) represent that the adjacent operations of the same job need to satisfy the priority constraint; Eqs. (4) and (5) denote that only one operation can be processed by the same machine at a time. Eq. (6) is also a machine constraint, which means that the

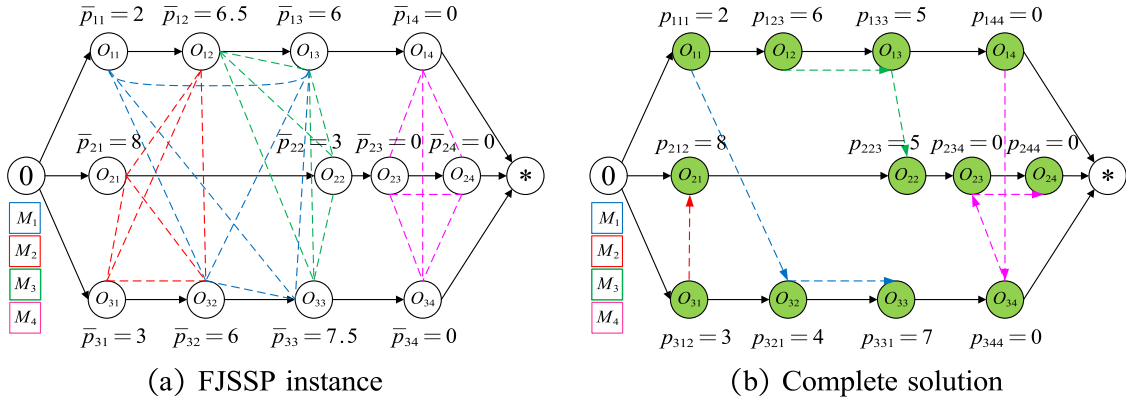


Fig. 1. Disjunctive graph representation of FJSSP after preprocessing.

Table 2

A 3×3 FJSSP instance.

Jobs	Operations	Optional machines and processing time		
		M_1	M_2	M_3
J_1	O_{11}	2	–	–
	O_{12}	–	7	6
	O_{13}	7	–	5
J_2	O_{21}	–	8	–
	O_{22}	–	–	3
J_3	O_{31}	–	3	–
	O_{32}	4	8	–
	O_{33}	7	–	8

same operation can only be processed by one machine at a time. The earliest completion time C_{ij}^E of O_{ij} on M_k is calculated as follows:

$$C_{ij}^E = S_{ij}^E + p_{ijk} \quad (7)$$

where, the calculation formula of S_{ij}^E is as follows:

$$S_{ij}^E = \max \{C^E[PJ(O_{ij})], C^E[PM(O_{ij})]\} \quad (8)$$

where, $PJ(O_{ij})$ indicates the predecessor of O_{ij} on the same job, and $PM(O_{ij})$ stands for the predecessor of O_{ij} on the same machine. $C^E[PJ(O_{ij})]$ and $C^E[PM(O_{ij})]$ represent the earliest completion time of the above two predecessor, respectively. If $PJ(O_{ij})$ or $PM(O_{ij})$ does not exist, S_{ij}^E is replaced by 0.

If machine is determined in advance for each operation in FJSSP, the problem is transformed into general JSSP. The disjunctive graph(DG) model (Błażewicz, Pesch, & Sterna, 2000; Mason & Oey, 2003) usually is used to describe JSSP. Correspondingly, FJSSP can also be described using a DG model. $DG = (\mathcal{N}, \mathcal{A}, \mathcal{E})$ is represented as a DG of FJSSP, where $\mathcal{N}$ represents the set of all operations containing two virtual nodes (a virtual start node 0 and a virtual end node *); $\mathcal{A}$ means the set of connecting arcs, which represents priority constraints between adjacent operations on the same job; $\mathcal{E}$ is the set of disjunctive arcs, and each disjunctive arc initially represents the bidirectional arcs of adjacent operations (or denoted as undirected states) on the same machine. During solving, the directions of disjunctive arcs between the scheduled nodes are determined. Table 2 shows an instance of FJSSP with 3 jobs and 3 machines.

To facilitate the calculation of the estimated completion time of each operation node via an array or tensor, we preprocess the instance so that each job has an equal number of operations. It can be seen from Table 2 that the number of operations corresponding to the jobs are 3, 2 and 3, respectively. To make the number of operations for all jobs is equal by padding some operation nodes, it is necessary to set a virtual machine M_4 and the processing time of any operation node on M_4 is 0. Fig. 1 shows a disjunction graph representation of

FJSSP after preprocessing. Above each node in Fig. 1(a) is the average processing time $\bar{p}_{ij}$ of the operation node on O_{ij} , and the disjunctive arcs of each color connect the operation nodes processed on the same machine. Fig. 1(b) stands for a complete solution. Above each node is the actual processing time of the operation node, and the direction of the disjunctive arc is determined on the same machine.

Since m of each instance in the public dataset is very close to n_{max} , we ensure that the number of operations for each job in an instance is equal by padding way, while making that the number of machines (adding some virtual machines) equal the number of operations for each job. To ensure that the number of operations for each job in the instance is equal, we need to consider the following two cases: (1) When m is greater than or equal to n_{max} and the number of operations of some jobs is less than n_{max} , we set a virtual machine M_{m+1} and make the number of operations for each job equal to $m + 1$ by padding some operation nodes processed on M_{m+1} ; (2) When m is less than n_{max} , we set a virtual machine M_{m+1} and make the number of operations for each job equal to n_{max} by padding some operation nodes processed on M_{m+1} . Note that if the difference between m and n_{max} is large, the number of operations for each job in an instance is guaranteed to be equal by padding some operations with only one virtual machine, regardless of whether the number of machines is equal to the number of operations for each job.

4. Proposed method

The proposed method mainly involves representation learning and policy learning. For representation learning, MLP as the state embedding network. For policy learning, the PPO algorithm (Schulman, Wolski, Dhariwal, Radford, & Klimov, 2017) under the actor-critic framework as training algorithm is used to learn the policy model. The actor-critic framework includes an actor network and a critic network, and the two networks are also implemented by MLP. The PPO algorithm is a new policy gradient algorithm that is easy to implement, and it has experienced the evolution process of the policy gradient algorithm (Sutton & Barto, 2018) and TRPO algorithm (Schulman, Levine, Abbeel, Jordan, & Moritz, 2015). Fig. 2 shows the overall framework of the proposed method.

4.1. Defining Markov decision process

Model FJSSP as an MDP model, and the MDP model mainly consists of a 5-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. $\mathcal{S}$ means state representation; $\mathcal{A}$ represents action space; $\mathcal{P}$ refers to state transition function, which is determined by FJSSP environment; $\mathcal{R}$ indicates reward function; γ is discount factor. In order to perform the state representation, the definition of action space and the design of reward function more intuitively, an example is used to show state transition process of FJSSP. Fig. 3 stands for an example of state transition, where the light green node

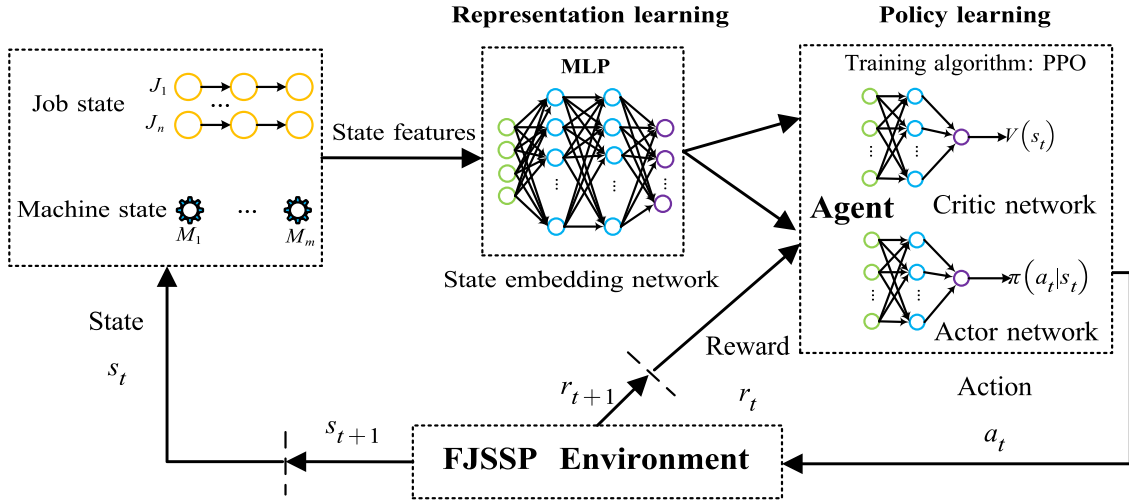


Fig. 2. The overall framework of the proposed method.

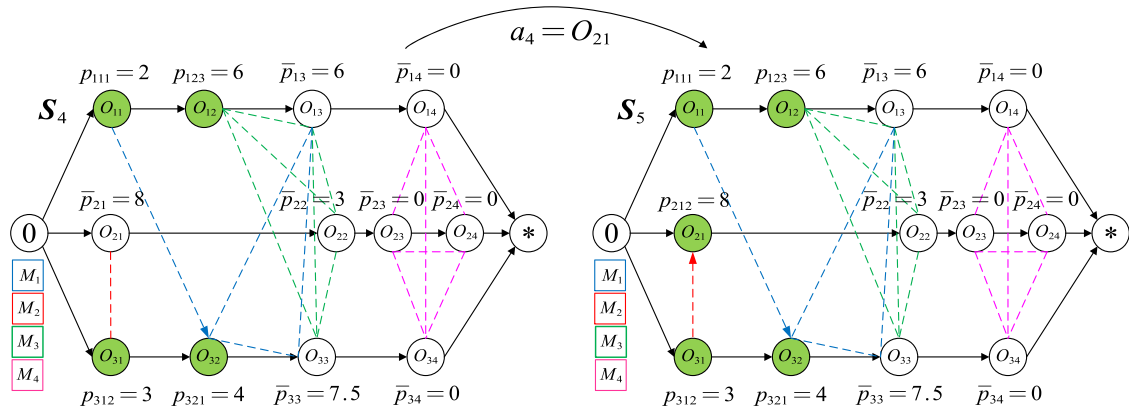


Fig. 3. An example of state transition.

represents the scheduled node, and the direction of the disjunction arc will be determined between adjacent scheduled nodes on the same machine. If an operation has been scheduled, the actual processing time is displayed above the operation node. Otherwise, it is the average processing time.

State. The state representation mainly reflects the state features of candidate actions in state s_t , where candidate actions consist of sub-action pairs composed of the candidate operation set $O_C(s_t)$ and the machine set. The four state features represented are as follows: (1) $p_{ijk}(s_t)$, which refers to the processing time of $O_{ij} \in O_C(s_t)$ on each machine. When the operation node is processed on an illegal machine, its processing time can be set to a negative value (e.g., -1), which indicates that the operation node is invalid on the selected machine. (2) $C_{LB}(O_{ij}, s_t)$, which means the lower bound of estimated completion time of $O_{ij} \in O_C(s_t)$ in state s_t . It is worth noting that $C_{LB}(O_{ij}, s_t)$ needs to be expanded to the form of a matrix $n \times (m+1)$ by row according to the number of machines $(m+1)$. According to Eqs. (7) and (8), its calculation formula is as follows:

$$C_{LB}(O_{ij}, s_t) = \max \{C^E[PJ(O_{ij}), s_t], C^E[PM(O_{ij}), s_t]\} + \bar{p}_{ij} \quad (9)$$

(3) $R_{LB}(O_{ij}, s_t)$, which represents the lower bound of estimated remaining processing time of job J_i in state s_t , assuming that $O_{ij} \in O_C(s_t)$ has been scheduled. Its calculation formula is as follows:

$$R_{LB}(O_{ij}, s_t) = C_{LB}(O_{in_i}, s_t) - C_{LB}(O_{ij}, s_t) \quad (10)$$

where, $C_{LB}(O_{in_i}, s_t)$ means the estimated completion time of the last operation of job J_i in state s_t . When O_{in_i} is an unscheduled node, its

lower bound is calculated recursively by considering only the priority constraint from its predecessor. Meanwhile, $R_{LB}(O_{ij}, s_t)$ also needs to be expanded to the form of a matrix $n \times (m+1)$ by row according to the number of machines $(m+1)$. (4) $S^E(O_{ij}, M_k, s_t)$, which indicates the earliest start time of $O_{ij} \in O_C(s_t)$ on each machine, assuming that an operation can be processed on an illegal machine. Its calculation formula is as follows:

$$S^E(O_{ij}, M_k, s_t) = \max \{C^E[PJ(O_{ij}), s_t], R^E(M_k, s_t)\} \quad (11)$$

where, $R^E(M_k, s_t)$ means the earliest release time of each machine M_k in state s_t . In the initial state, the earliest release time of M_k is set to 0, that is, $R^E(M_k, s_0) = 0$; During solving, the earliest release time of each machine is continuously updated under the condition of satisfying various constraints. When O_{ij} is the first operation of the job J_i , that is, $PJ(O_{ij})$ does not exist, then $C^E[PJ(O_{ij}), s_t] = 0$.

According to the example of state transition in Fig. 3, the set of candidate operation nodes is $\{O_{13}, O_{21}, O_{33}\}$ in state s_4 . From Table 2, we can know the processing time of $O_{ij} \in O_C(s_t)$ on each machine, and recursively calculate their estimated completion time by combining Eq. (9). Fig. 4 shows an example of state representation.

Action. At decision step t , action space A_t is composed of candidate actions. Each candidate action $a_t \in A_t$ is represented as a sub-action pair (candidate operation action a_t^o and candidate machine action a_t^m). Each job provides a candidate operation action, so that the total number of candidate operation actions is equal to n . Assume that each candidate operation action can be processed on any machine, so the total number of candidate actions is $n \times (m+1)$, where the number of candidate machine actions is always equal to $m+1$. It should be noted

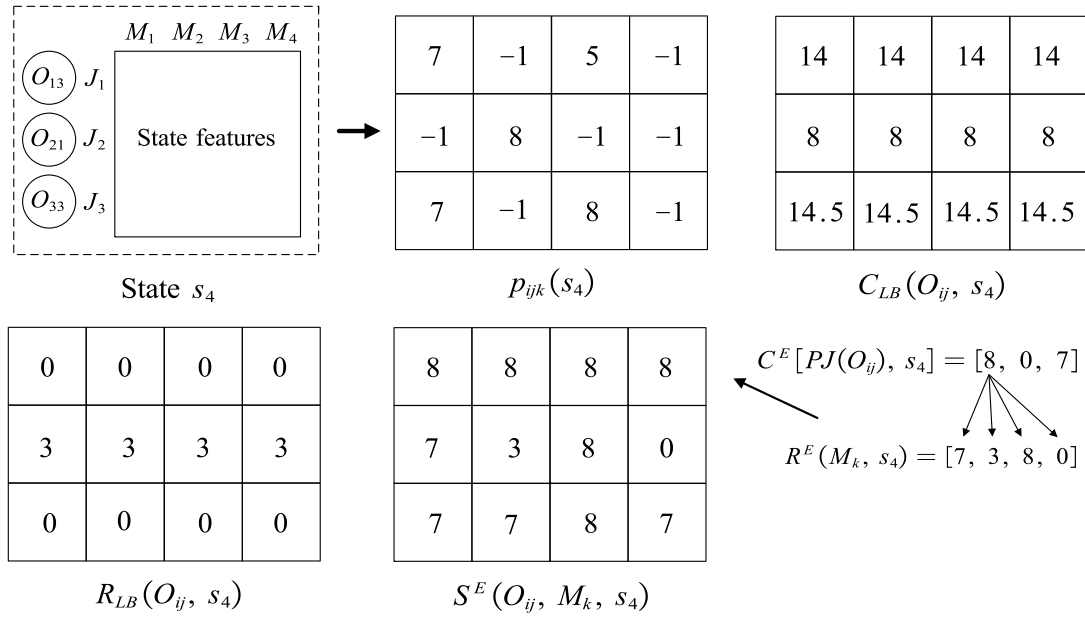
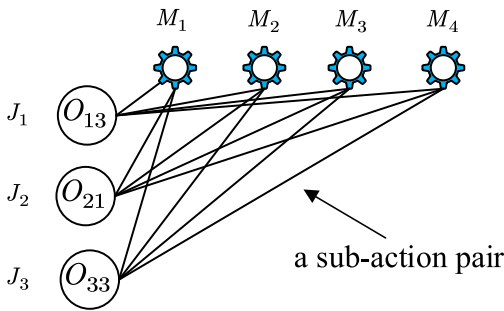


Fig. 4. An example of state representation.



The set of candidate actions in state s_4 :

$$\left\{ \begin{array}{l} O_{13} - M_1, O_{13} - M_2, O_{13} - M_3, O_{13} - M_4, \\ O_{21} - M_1, O_{21} - M_2, O_{21} - M_3, O_{21} - M_4, \\ O_{33} - M_1, O_{33} - M_2, O_{33} - M_3, O_{33} - M_4 \end{array} \right\}$$

Fig. 5. An example of action space.

that at each decision step t , invalid actions in the candidate actions need to be masked. Invalid action masking is divided into the following two situations: (1) When a candidate operation is processed on an illegal machine, the candidate action composed of this sub-action pair will be masked. (2) When a candidate operation is the last operation of a job and the operation has been scheduled, the candidate action composed of this sub-action pair will be masked. Also taking state s_4 in Fig. 3 as an example, Fig. 5 shows an example of action space.

Reward. The definition of reward function is the same as Zhang et al. (2020), which is also $r_t = C_{\max}(s_t) - C_{\max}(s_{t+1})$, where $C_{\max}(s_t) = \max_{1 \leq i \leq n} \{C_{LB}(O_{in_i}, s_t)\}$. It can be seen that the computation of r_t is closely related to $C_{LB}(O_{in_i}, s_t)$. Assume that an instance does not satisfy the condition that the number of operations for each job is equal, it is necessary to pad some operation nodes so that the total number of operations is T in the instance. Note that the total number of operations in an instance is equal to the number of decision steps in an episode. When the discount factor $\gamma = 1$, the cumulative reward

$G = \sum_{t=0}^T r_t = C_{\max}(s_0) - C_{\max}(s_T)$, where $C_{\max}(s_0)$ is a constant, so maximizing G and minimizing $C_{\max}$ is equivalent.

4.2. Training

The state embedding network is implemented by a 3-layer MLP (excluding the input layer), where the input layer contains 4 neurons, the hidden layer contains 128 neurons, and the output layer contains 64 neurons. The actor network is implemented by a 2-layer MLP (excluding the input layer), where the input layer contains 128 neurons, the hidden layer contains 64 neurons, and the output layer contains 1 neuron; the critic network is also implemented by a 2-layer MLP (excluding the input layer), where the input layer contains 64 neurons, the hidden layer contains 64 neurons, and the output layer contains 1 neuron. The activation function between the network layers is "tanh". The number of episodes is N , which is also equivalent to the size of the training dataset; the performance of the policy model is verified after every V episodes; K represents the number of policy updates per episode. The learning rate is expressed as lr .

The PPO algorithm updates the network model parameters by gradient ascent. The "surrogate" objective function $L_t^{CLIP}(\theta)$ (Schulman et al., 2017) can be expressed as follows:

$$L_t^{CLIP}(\theta) = \hat{E}_t [\min(R_t(\theta)\hat{A}_t, \text{clip}(R_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (12)$$

where,

$$R_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad (13)$$

is the ratio of new and old probabilities; $\hat{A}_t$ is the advantage function; ϵ is the clipping coefficient. In addition, two additional terms, an entropy term and a mean squared error term, need to be added to the objective function. The total loss function (Schulman et al., 2017) is written as follows:

$$L_t(\theta, \phi) = c_p L_t^{CLIP}(\theta) + c_e L_t^E(\theta) - c_v L_t^V(\phi) \quad (14)$$

where, $L_t^E(\theta)$ is the entropy item, and $L_t^V(\phi)$ is the mean square error item; c_p , c_e and c_v are policy loss coefficient, entropy loss coefficient and critic loss coefficient, respectively. The pseudocode of updating network model parameters via PPO algorithm is shown in Algorithm 1.

Algorithm 1: Pseudocode for updating the network model parameters via PPO algorithm.

Input: Actor network π_θ with parameter β , behavior actor network $\pi_{\theta_{old}}$ with $\theta_{old} = \theta$, and critic network v_ϕ with ϕ . N , K , V .

Output: θ, ϕ

```

1 for  $n = 1, 2, \dots, N$  do
  /*  $n$  means an episode. */
2   for  $t = 1, 2, \dots, T$  do
3     while  $s_t$  is not terminal do
4       Collect the experience  $\langle s_t, a_t, r_t, s_{t+1} \rangle$ .
5     end
6     if  $s_t$  is terminal then
7       break
8     end
9   endfor
10  for  $k = 1, 2, \dots, K$  do
11    The experience of an episode is collected to calculate
    the loss  $\mathcal{L}_t(\theta, \phi)$ , and the parameters  $\theta$  and  $\phi$  are
    updated.
12  endfor
13  if  $n\%V = 0$  then
14    A performance verification of the policy model.
15  end
16 endfor

```

5. Experiment

5.1. Experimental settings

5.1.1. Configuration

The hardware platform of this paper: Intel(R) Core(TM) i7-9750H CPU@2.60 GHz 2.59 GHz. The GPU is NVIDIA GeForce GTX 1660 Ti with a memory size of 6144MB. The software platform: Python version is 3.7.0. PyTorch is available in version 1.6.0, and default parameters settings in PyTorch follow [Paszke et al. \(2019\)](#). The hyperparameter settings of the PPO algorithm are shown in [Table 3](#).

5.1.2. Datasets

In this paper, the datasets contain three types: the training datasets, the validation dataset and the test datasets. The training datasets and the validation dataset are artificially synthesized. The test datasets are public datasets that are used to compare the performance of different algorithms. The details of the datasets are shown in [Table 4](#). It contains the training datasets with different sizes that are the same scale (10×5) as the validation dataset. The test datasets are some public datasets with various scales. We set the size of the training datasets to 300, 400, 500 and 600, respectively, to test how it affects the performance of the policy model. Since the training datasets and the validation dataset are synthetic, we set the seed of the training datasets to 200 and the seed of the validation dataset to 100 for ease of reproduction.

5.1.3. Evaluation metric

The optimization objective is to minimize the makespan in this paper, and the relative percentage deviation (RPD) is used as the metric ([Pan, Wang, Wang, & Lu, 2021](#)) to compare the optimization objective performance between different methods, which is also known as the optimal gap. It is calculated as follows:

$$RPD = \frac{C'_{\max} - C^*_{\max}}{C^*_{\max}} \times 100\%, \quad (15)$$

where, $C'_{\max}$ is the $C_{\max}$ obtained by an algorithm, and $C^*_{\max}$ is the optimal solution or approximate optimal solution obtained by the exact method.

Table 3

The hyperparameter settings of the PPO algorithm.

Hyperparameters	Values
Number of episodes N	300, 400, 500, 600
Number of policy updates K	1
Number of interval episodes per validation V	3, 4, 5, 6
Learning rate lr	1×10^{-4}
Policy loss coefficient c_p	2
Entropy loss coefficient c_e	0.01
Critic loss coefficient c_v	1
Discount factor γ	1
Clipping coefficient ϵ	0.2

Table 4

Details of the dataset.

Datasets	Source	Seed	Range	Size
Training dataset	Synthetic	200	$1 \leq p \leq 99$	300, 400, 500, 600
Validation dataset	Synthetic	100	$1 \leq p \leq 99$	100
Test dataset 1	Barnes	–	$1 \leq p \leq 99$	21
Test dataset 2	Brandimarte	–	$1 \leq p \leq 20$	10
Test dataset 3	Dauzere	–	$1 \leq p \leq 100$	18
Test dataset 4	Hurink(rdata)	–	$1 \leq p \leq 99$	40
	Hurink(edata)	–	$1 \leq p \leq 99$	40
	Hurink(vdata)	–	$1 \leq p \leq 99$	40

5.1.4. Baselines

The proposed method is compared with nine baselines, including four meta-heuristic methods, three well-known composite PDRs and two current state-of-the-art models. The four meta-heuristic methods include Improved Jaya Algorithm (IJA) ([Caldeira & Gnanavelbabu, 2019](#)), Regular GA (RegGA) ([Rooyani & Defersha, 2019](#)), Two-Stage Genetic Algorithm (2SGA) ([Rooyani & Defersha, 2019](#)), and Self-learning Genetic Algorithm (SLGA) ([Chen et al., 2020](#)). The three composite PDRs are FIFO+EET, MWKR+EET and MOPNR+EET. The two current state-of-the-art models are ([Lei et al., 2022](#); [Song et al., 2022](#)).

- Z_{ij} : the priority index of O_{ij} ;
- FIFO: index of operation with the earliest ready time.
- MWKR: $\max Z_{ij} = \sum_j^{n_i} \bar{p}_{ij}$.
- MOPNR: $\max Z_{ij} = n_i - j + 1$.
- EET: index of machine with earliest end time.

5.2. Experimental results

5.2.1. Effect of training datasets with different sizes

In order to verify whether the policy model is effective during training, we set the size of the training dataset to 300, 400, 500, and 600, respectively. [Fig. 6](#) shows the convergence of the policy models under different sizes of the training datasets. In [Fig. 6](#), the policy models show good convergence under different sizes of training datasets, which reflect the effectiveness of the policy models. Although the policy models show good convergence on the validation dataset, the generalization of these policy models need to be further evaluated on public datasets.

[Table 5](#) shows average gap and average time of these policy models on the public datasets. In [Table 5](#), the performance of these policy models are relatively stable in terms of average gap and average time. When the size of the training dataset is $N = 500$, the performance of the policy model is relatively better on the whole. When the size of the training dataset is $N = 300$, the policy model performs better on Barnes dataset. When the size of the training dataset is $N = 400$, the policy model performs better on Brandimarte dataset. It can be seen that during training, the performance of the policy model will be affected by different sizes of training datasets, which will affect the generalization of the policy model to some extent. Research on the generalization mechanism of RL is still in its infancy, and some works ([Cobbe, Klimov, Hesse, Kim, & Schulman, 2019](#); [Ghosh et al., 2021](#); [Witty et al., 2021](#)) present partial insights into the generalization mechanisms.

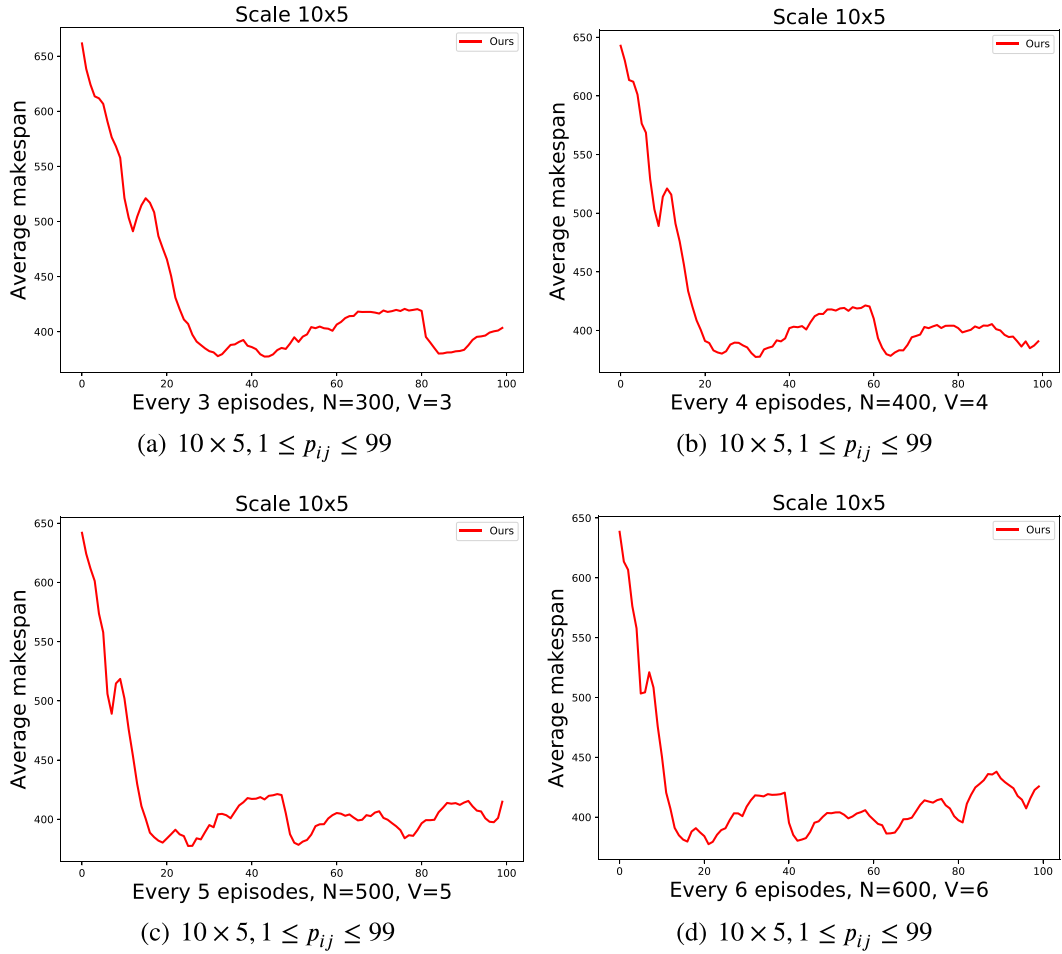


Fig. 6. The training curves of the policy models under different sizes of training datasets.

Table 5

Average gap and average time of policy models on public datasets under different sizes of training datasets.

10×5	$N = 300, V = 3$		$N = 400, V = 4$		$N = 500, V = 5$		$N = 600, V = 6$	
	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)
Barnes	13.83%	0.391	17.88%	0.392	15.44%	0.402	16.39%	0.397
Brandimarte	17.00%	0.406	13.24%	0.406	17.84%	0.404	16.47%	0.403
Dauzere	11.60%	0.804	14.91%	0.798	11.16%	0.809	11.02%	0.808
Hurink(rdata)	13.41%	0.274	13.98%	0.273	12.09%	0.275	12.66%	0.275
Hurink(edata)	15.69%	0.274	15.95%	0.274	15.54%	0.271	16.57%	0.273
Hurink(vdata)	6.50%	0.278	6.30%	0.272	5.37%	0.272	6.51%	0.276

Table 6

Comparison of average gap and average time on public datasets with various baselines.

Methods	Barnes		Brandimarte		Dauzere		Hurink(rdata)		Hurink(edata)		Hurink(vdata)	
	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)
IJA	2.40%	21.03	8.50%	15.43	6.10%	180.8	3.90%	19.72	4.60%	15.24	2.70%	17.85
RegGA	–	–	8.39%	280.10	–	–	–	–	–	–	3.20%	191.40
2SGA	–	–	3.17%	57.60	–	–	–	–	–	–	0.39%	51.43
SLGA	–	–	6.21%	283.28	–	–	–	–	–	–	–	–
FIFO+EET	27.91%	0.019	28.98%	0.017	15.00%	0.036	17.38%	0.018	19.89%	0.016	7.14%	0.019
MWKR+EET	54.71%	0.018	39.50%	0.019	24.69%	0.036	26.60%	0.018	44.68%	0.018	8.96%	0.020
MOPNR+EET	50.66%	0.017	43.39%	0.018	32.17%	0.038	26.47%	0.019	43.67%	0.018	13.14%	0.019
Song et al. (2022)	17.88%	1.516	30.04%	1.305	8.88%	2.716	11.02%	1.421	16.66%	1.424	4.41%	1.405
Lei et al. (2022)	28.96%	2.048	13.59%	2.054	15.68%	3.917	16.51%	1.591	23.01%	1.558	6.96%	1.544
Ours	13.83%	0.391	13.24%	0.406	11.02%	0.808	12.09%	0.275	15.54%	0.271	5.37%	0.272

5.2.2. Comparison with various baselines

Table 6 shows performance comparison with various baselines. In Table 6, based on the open source codes provided by Lei et al. (2022) and Song et al. (2022), we replicate them on our own device with

small scale 10×5 and save multiple best models for testing, and the best experimental results are close to those in Lei et al. (2022) and Song et al. (2022). Notably, for a fair comparison, we compare with the results obtained in the greedy strategy by Song et al. (2022). As

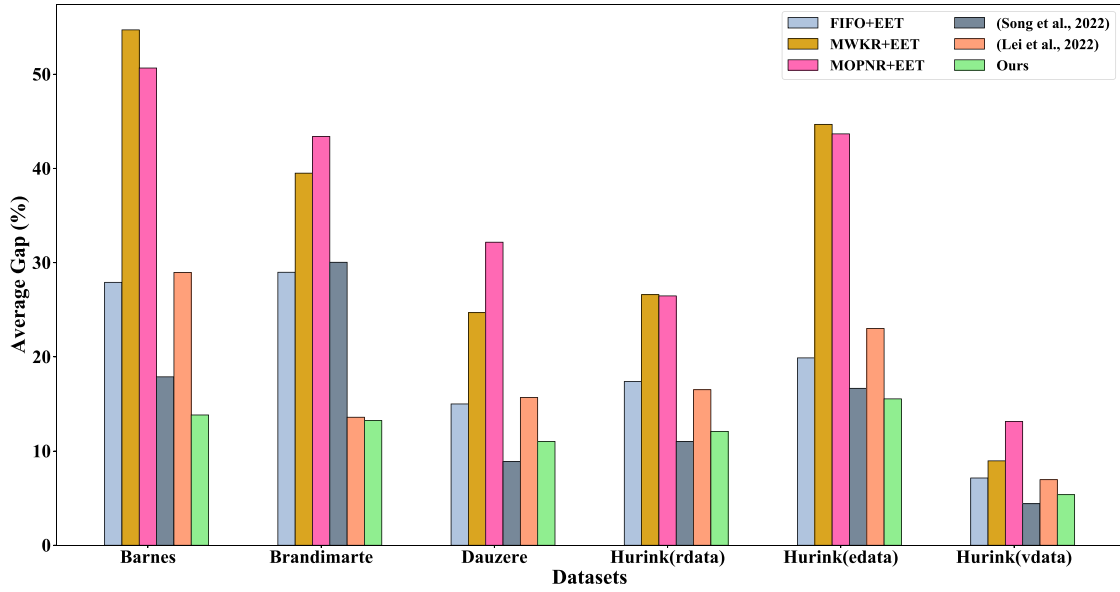


Fig. 7. Comparison chart with other learning-based methods and composite PDRs.

can be seen from Table 6, the average gap of the proposed method on most public datasets is close to Lei et al. (2022) and Song et al. (2022), and even performs better on some datasets, such as Barnes dataset and Hurink(edata) dataset. Meanwhile, the average time is the lowest on all public datasets. On the one hand, since the new state representation can directly reflect the state information of the candidate actions, it is conducive to improving the agent's decision-making ability. On the other hand, a lightweight MLP as state embedding network rather than a complex GNN. In addition, the new action space is able to solve both subproblems of the FJSSP simultaneously in the case of having a single action space, which means that only one policy model is trained. These measures are beneficial to reducing computational complexity of the algorithm to some extent. Song et al. (2022) shows better optimization ability on Dauzere dataset; however, it shows weaker optimization ability on Brandimarte dataset. Lei et al. (2022) shows optimization ability as good as our method on Brandimarte dataset. However, it shows weak optimization ability on other datasets, and even performs poorly on some public datasets, such as the Barnes and Hurink(edata) datasets. Compared with well-known composite PDRs, these PDRs take the least time and demonstrate good applicability, but the optimization ability is relatively weak. This is because these PDRs are easy to implement, but shortsighted. Compared with meta-heuristic methods, these meta-heuristic methods have stronger optimization ability, but their computational complexity is high. This is because these meta-heuristic methods improve the quality of the solutions by continuously iterating the search based on providing some initial solutions. The proposed method belongs to the learning-based method, and experimental results show that the proposed method achieves a better compromise in terms of optimization ability and applicability. It can be said that the exact method has the strongest optimization ability, but poor applicability; the PDR has the strongest applicability, but poor optimization ability; the learning-based method is a compromise method between optimization ability and applicability.

We add the visualization of the comparison results to better demonstrate the effectiveness of the proposed method. Fig. 7 shows the comparison chart with other learning-based methods and composite PDRs. As can be seen from Fig. 7, the optimization ability of the proposed method is very close to the current state-of-the-art models on most public datasets, and its performance is relatively stable on all public datasets. However, the optimization ability of the current state-of-the-art models fluctuate greatly on some public datasets.

6. Conclusions

In this paper, we solve FJSSP via DRL. First, we propose a new DRL framework to implement representation learning and policy learning. A lightweight MLP as state embedding network is used to extract the state information of candidate actions rather than a complex GNN, which reduces the computational complexity of the algorithm to some extent. Next, we define a new state representation and a new action space. The new state representation contains four state features, which can effectively reflect the state features of candidate actions and improve the decision-making ability of the agent. The new action space composed of sub-action pairs can solve two sub-problems of FJSSP simultaneously with only one action space, so only one policy model needs to be trained. Finally, we train the policy model on training dataset with small scale 10×5 , and employ the generalization of the policy model to solve other scale problems. Compared with the current state-of-the-art models, the proposed method not only has good optimization ability, but also has lower computational complexity, showing a better compromise in terms of optimization ability and applicability. In future work, we will consider using DRL method to solve multi-objective FJSSP and dynamic FJSSP.

Funding statement

This research was funded in part by the Scientific and technological innovation 2030 major project under Grant 2022ZD0115800, the Xinjiang Uygur Autonomous Region Tianshan Excellence, China Project under grant 2022TSYCLJ0036, the National Science Foundation of China under Grant U1903213, the Key Project of the National Natural Science Foundation of China under Grant 61936009, Natural Science Foundation of Xinjiang Uygur Autonomous Region under Grant 2022D01C82, and Xinjiang Uygur Autonomous Region Postgraduate Research and Innovation Project under Grant XJ2022G025.

CRediT authorship contribution statement

Erdong Yuan: Writing – original draft, Software, Conceptualization. **Liejun Wang:** Supervision, Funding acquisition, Writing – review & editing. **Shuli Cheng:** Validation, Writing – review & editing, Formal analysis. **Shiji Song:** Funding acquisition, Resources, Visualization. **Wei Fan:** Resources, Formal analysis, Visualization. **Yongming Li:** Resources, Formal analysis, Visualization.

Table 7

Test results of different methods on the Barnes dataset.

Instances	UB	FIFO+EET	MWKR+EET	MOPNR+EET	Song et al. (2022)	Lei et al. (2022)	Ours
mt10x	918	1223	1253	1358	1096	1176	1084
mt10xx	918	1256	1253	1374	1069	1176	1084
mt10xxx	918	1250	1253	1416	1069	1176	1084
mt10xy	906	1170	1252	1391	1090	1148	1055
mt10xyz	847	1058	1111	1292	1003	1180	1020
mt10c1	928	1207	1252	1353	1159	1240	1096
mt10cc	910	1178	1121	1389	1100	1226	1068
setb4x	925	1140	1664	1322	1118	1210	1022
setb4xx	925	1149	1691	1349	1119	1210	1015
setb4xxx	925	1140	1664	1282	1119	1210	1040
setb4xy	916	1125	1664	1372	1081	1111	1019
setb4xyz	905	1082	1664	1355	1149	1095	1000
setb4c9	919	1178	1589	1379	1073	1225	1037
setb4cc	909	1234	1532	1363	1016	1180	993
seti5x	1201	1445	1756	1918	1377	1592	1363
seti5xx	1199	1517	1731	1700	1377	1592	1344
seti5xxx	1197	1590	1755	1852	1373	1592	1340
seti5xy	1136	1404	1732	1728	1299	1438	1271
seti5xyz	1125	1464	1732	1592	1250	1298	1254
seti5c12	1174	1523	1953	1899	1390	1467	1346
seti5cc	1136	1436	1732	1921	1299	1438	1271
Average gap	0.00%	27.91%	54.71%	50.66%	17.88%	28.96%	13.83%

Table 8

Test results of different methods on the Brandimarte dataset.

Instances	UB	FIFO+EET	MWKR+EET	MOPNR+EET	Song et al. (2022)	Lei et al. (2022)	Ours
mk01	40*	49	51	57	49	47	44
mk02	26	43	41	41	43	30	31
mk03	204*	205	210	234	216	204	211
mk04	60*	78	99	90	75	76	78
mk05	173	184	202	211	190	178	183
mk06	58	92	112	114	103	79	74
mk07	144	214	215	219	212	152	156
mk08	523*	541	579	631	523	541	524
mk09	307	350	384	397	349	335	326
mk10	198	278	291	294	264	236	241
Average gap	0.00%	28.98%	39.50%	43.39%	30.04%	13.59%	13.24%

Table 9

Test results of different methods on the Dauzere dataset.

Instances	UB	FIFO+EET	MWKR+EET	MOPNR+EET	Song et al. (2022)	Lei et al. (2022)	Ours
01a	2518	3138	3352	3545	2847	3109	2947
02a	2231	2416	2753	3084	2478	2603	2495
03a	2229	2359	2456	2650	2294	2366	2320
04a	2503*	3109	3188	3551	2856	3093	3037
05a	2216	2525	2684	2887	2473	2573	2496
06a	2203	2410	2478	2496	2262	2508	2286
07a	2283	2897	3628	3813	2736	2857	2820
08a	2069	2321	2622	2639	2191	2386	2259
09a	2066	2170	2313	2391	2097	2209	2155
10a	2291	2786	3386	3501	2635	2805	2721
11a	2063	2401	2512	2804	2227	2356	2206
12a	2034	2203	2228	2424	2090	2186	2157
13a	2260	3044	3320	3500	2739	2811	2755
14a	2167	2347	2510	2660	2218	2353	2277
15a	2167	2325	2308	2471	2193	2385	2202
16a	2255	2821	3336	3313	2723	2861	2680
17a	2141	2356	2481	2653	2224	2365	2309
18a	2137	2308	2289	2458	2185	2363	2216
Average gap	0.00%	15.00%	24.69%	32.17%	8.88%	15.68%	11.02%

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Appendix

In the appendix, we present the test results of different methods on the Barnes dataset, Brandimarte dataset, Dauzere dataset, Hurink (rdata) dataset, Hurink(edata) dataset and Hurink(vdata) dataset through [Tables 7, 8, 9, 10, 11](#) and [12](#), respectively. Noted that the “UB” column means the upper bound of the optimal solution for the instance, and “*” indicates that the solution is optimal.

Table 10

Test results of different methods on the Hurink(rdata) dataset.

Instances	UB	FIFO+EET	MWKR+EET	MOPNR+EET	Song et al. (2022)	Lei et al. (2022)	Ours
la01	571	689	709	687	616	690	663
la02	530	668	694	695	598	648	666
la03	478	538	623	565	507	528	525
la04	502*	601	633	624	533	531	573
la05	457*	502	540	507	504	516	500
la06	799*	857	912	848	850	850	856
la07	750	798	846	908	787	786	825
la08	765*	821	960	829	804	803	820
la09	853*	924	1068	940	875	866	885
la10	804*	846	973	920	832	885	870
la11	1071*	1123	1157	1156	1135	1105	1131
la12	936*	1004	1050	1086	957	988	977
la13	1038*	1089	1120	1127	1107	1117	1051
la14	1070*	1159	1161	1201	1107	1152	1110
la15	1090	1264	1202	1282	1228	1258	1236
la16	717*	913	984	968	794	851	821
la17	646*	770	794	825	731	892	720
la18	666*	891	920	888	814	744	766
la19	700	882	885	1023	884	952	812
la20	756*	882	1117	1001	943	964	878
la21	835	1055	1096	1146	989	1073	967
la22	760	1125	1072	1132	857	1001	893
la23	842	1035	1137	1146	956	1066	1010
la24	808	1001	1095	1085	958	966	928
la25	791	1063	1172	1130	895	964	939
la26	1061	1229	1292	1406	1173	1237	1175
la27	1091	1257	1314	1482	1225	1276	1214
la28	1080	1221	1343	1422	1199	1320	1175
la29	998	1250	1387	1273	1101	1219	1111
la30	1078	1230	1361	1382	1227	1371	1275
la31	1521	1644	1829	1840	1591	1624	1636
la32	1659	1824	1913	1931	1798	1849	1765
la33	1499	1605	1891	1700	1568	1663	1542
la34	1536	1702	1747	1767	1577	1599	1686
la35	1550	1737	1773	1764	1692	1706	1706
la36	1030	1320	1372	1411	1296	1296	1229
la37	1077	1299	1488	1528	1210	1285	1294
la38	962	1174	1389	1516	1128	1188	1107
la39	1024	1329	1452	1455	1174	1229	1228
la40	970	1167	1435	1392	1098	1200	1108
Average gap	0.00%	17.38%	26.60%	26.47%	11.02%	16.51%	12.09%

Table 11

Test results of different methods on the Hurink(edata) dataset.

Instances	UB	FIFO+EET	MWKR+EET	MOPNR+EET	Song et al. (2022)	Lei et al. (2022)	Ours
la01	609*	722	866	882	688	712	658
la02	655*	797	982	992	851	875	809
la03	550*	691	692	988	657	714	660
la04	568*	664	872	836	660	711	669
la05	503*	593	737	599	530	621	593
la06	833*	847	1030	955	926	946	951
la07	762*	969	1065	1063	910	1010	896
la08	845*	949	1267	1162	969	932	947
la09	878*	934	1229	1085	1046	964	946
la10	866*	934	1167	1032	883	1016	886
la11	1103	1190	1451	1274	1172	1285	1213
la12	960*	1032	1282	1154	1001	1143	1065
la13	1053*	1150	1392	1270	1215	1181	1164
la14	1123*	1306	1447	1487	1191	1177	1234
la15	1111*	1308	1541	1389	1301	1362	1297
la16	892*	1087	1351	1414	1059	1046	1051
la17	707*	813	1039	1016	781	949	773
la18	842*	952	1089	1153	944	970	934
la19	796*	981	1100	1234	950	1001	972
la20	857*	1102	1317	1199	1007	1102	1004
la21	1017	1282	1363	1594	1243	1262	1256
la22	882	1234	1310	1208	977	1242	1070
la23	950*	1156	1396	1394	1131	1184	1137
la24	909	1186	1527	1360	1112	1204	1086
la25	941	1224	1401	1629	1139	1224	1102
la26	1125	1372	1647	1674	1338	1478	1348

(continued on next page)

Table 11 (continued).

Instances	UB	FIFO+EET	MWKR+EET	MOPNR+EET	Song et al. (2022)	Lei et al. (2022)	Ours
la27	1186	1488	1863	1916	1485	1514	1423
la28	1149	1464	1769	1609	1441	1375	1344
la29	1118	1394	1741	1807	1353	1345	1359
la30	1204	1550	1846	1863	1459	1471	1410
la31	1539	1836	2170	2153	1720	1936	1755
la32	1698*	1967	2530	2490	1962	2043	1904
la33	1547*	1863	2396	2228	1776	1811	1785
la34	1604	1872	2183	2349	1821	1917	1817
la35	1736*	1970	2547	2498	1967	1996	2080
la36	1162	1518	1675	1775	1334	1638	1295
la37	1397	1602	1987	2177	1650	1645	1531
la38	1144	1376	1810	1690	1507	1355	1370
la39	1184	1490	1757	1884	1547	1659	1412
la40	1150	1447	1877	1798	1353	1422	1311
Average gap	0.00%	19.89%	44.68%	43.67%	16.66%	23.01%	15.54%

Table 12
Test results of different methods on the Hurink(vdata) dataset.

Instances	UB	FIFO+EET	MWKR+EET	MOPNR+EET	Song et al. (2022)	Lei et al. (2022)	Ours
la01	570*	665	660	642	609	618	612
la02	529*	565	662	587	582	583	601
la03	477*	546	544	595	517	533	533
la04	502*	527	598	659	554	555	556
la05	457*	492	578	614	493	563	525
la06	799*	881	890	938	859	837	888
la07	749*	810	816	811	785	777	802
la08	765*	830	848	853	805	838	783
la09	853*	880	931	939	862	891	887
la10	804*	837	867	871	839	825	843
la11	1071*	1135	1153	1117	1114	1120	1119
la12	936*	970	1015	1014	967	951	977
la13	1038*	1078	1084	1136	1076	1072	1082
la14	1070*	1083	1113	1119	1099	1115	1088
la15	1089*	1122	1111	1152	1104	1154	1110
la16	717*	809	767	726	753	768	727
la17	646*	668	659	728	658	700	648
la18	663*	671	680	704	663	676	663
la19	617*	675	647	757	654	697	639
la20	756*	798	766	807	756	772	766
la21	804	920	926	1043	839	894	891
la22	736	804	817	857	812	834	809
la23	815	871	895	957	868	863	872
la24	775	865	879	904	803	872	873
la25	756	936	877	926	809	893	840
la26	1054	1142	1145	1165	1073	1087	1100
la27	1084	1154	1151	1233	1133	1185	1131
la28	1070	1157	1171	1191	1100	1135	1106
la29	994	1091	1178	1138	1027	1088	1076
la30	1069	1143	1223	1215	1106	1138	1143
la31	1520*	1605	1591	1618	1565	1596	1575
la32	1658*	1705	1737	1781	1714	1733	1726
la33	1497*	1541	1588	1574	1529	1554	1531
la34	1537*	1583	1582	1645	1574	1588	1594
la35	1549*	1620	1600	1681	1612	1596	1575
la36	948*	1015	976	1106	993	964	973
la37	986*	1082	1006	1155	1069	1066	1024
la38	943*	952	981	1069	945	979	943
la39	922*	972	1026	1054	951	984	942
la40	955*	985	965	1066	985	971	967
Average gap	0.00%	7.14%	8.96%	13.14%	4.41%	6.96%	5.37%

References

Bagheri, A., Zandieh, M., Mahdavi, I., & Yazdani, M. (2010). An artificial immune algorithm for the flexible job-shop scheduling problem. *Future Generation Computer Systems*, 26(4), 533–541.

Barnes, J., & Chambers, J. (1996). Flexible job shop scheduling by tabu search. In *Graduate program in operations and industrial engineering: Technical Report Series, ORP96-09*, The University of Texas at Austin.

Błażewicz, J., Pesch, E., & Serna, M. (2000). The disjunctive graph machine representation of the job shop scheduling problem. *European Journal of Operational Research*, 127(2), 317–331.

Brammer, J., Lutz, B., & Neumann, D. (2022). Permutation flow shop scheduling with multiple lines and demand plans using reinforcement learning. *European Journal of Operational Research*, 299(1), 75–86.

Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3), 157–183.

Caldeira, R. H., & Gnanavelbabu, A. (2019). Solving the flexible job shop scheduling problem using an improved jaya algorithm. *Computers & Industrial Engineering*, 137, Article 106064.

Chen, R., Yang, B., Li, S., & Wang, S. (2020). A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Computers & Industrial Engineering*, 149, Article 106778.

Chiang, T.-C., & Fu, L.-C. (2007). Using dispatching rules for job shop scheduling with due date-based objectives. *International Journal of Production Research*, 45(14), 3245–3262.

Cobbe, K., Klimov, O., Hesse, C., Kim, T., & Schulman, J. (2019). Quantifying generalization in reinforcement learning. In *International conference on machine learning* (pp. 1282–1289). PMLR.

- Dauzère-Pérès, S., & Paulli, J. (1997). An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search. *Annals of Operations Research*, 70, 281–306.
- Doh, H.-H., Yu, J.-M., Kim, J.-S., Lee, D.-H., & Nam, S.-H. (2013). A priority scheduling approach for flexible job shops with multiple process plans. *International Journal of Production Research*, 51(12), 3748–3764.
- Du, Y., Li, J.-q., Chen, X.-l., Duan, P.-y., & Pan, Q.-k. (2022). Knowledge-based reinforcement learning and estimation of distribution algorithm for flexible job shop scheduling problem. *IEEE Transactions on Emerging Topics in Computational Intelligence*.
- Fattahi, P., Saidi Mehrabad, M., & Jolai, F. (2007). Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *Journal of Intelligent Manufacturing*, 18, 331–342.
- Gao, K., Cao, Z., Zhang, L., Chen, Z., Han, Y., & Pan, Q. (2019). A review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems. *IEEE/CAA Journal of Automatica Sinica*, 6(4), 904–916.
- Ghosh, D., Rahme, J., Kumar, A., Zhang, A., Adams, R. P., & Levine, S. (2021). Why generalization in rl is difficult: Epistemic pomdps and implicit partial observability. *Advances in Neural Information Processing Systems*, 34, 25502–25515.
- Han, W., Guo, F., & Su, X. (2019). A reinforcement learning method for a hybrid flow-shop scheduling problem. *Algorithms*, 12(11), 222.
- Han, B.-A., & Yang, J.-J. (2020). Research on adaptive job shop scheduling problems based on dueling double DQN. *IEEE Access*, 8, 186474–186495.
- Haupt, R. (1989). A survey of priority rule-based scheduling. *Operations-Research-Spektrum*, 11(1), 3–16.
- Hurink, J., Jurisch, B., & Thole, M. (1994). Tabu search for the job-shop scheduling problem with multi-purpose machines. *Operations-Research-Spektrum*, 15, 205–215.
- Karthikeyan, S., Asokan, P., Nickolas, S., & Page, T. (2015). A hybrid discrete firefly algorithm for solving multi-objective flexible job shop scheduling problems. *International Journal of Bio-Inspired Computation*, 7(6), 386–401.
- Lei, K., Guo, P., Zhao, W., Wang, Y., Qian, L., Meng, X., et al. (2022). A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem. *Expert Systems with Applications*, 205, Article 117796.
- Lin, C.-C., Deng, D.-J., Chih, Y.-L., & Chiu, H.-T. (2019). Smart manufacturing scheduling with edge computing using multiclass deep Q network. *IEEE Transactions on Industrial Informatics*, 15(7), 4276–4284.
- Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*, 91, Article 106208.
- Luo, S., Zhang, L., & Fan, Y. (2021). Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. *Computers & Industrial Engineering*, 159, Article 107489.
- Mason, S. J., & Oey, K. (2003). Scheduling complex job shops using disjunctive graphs: a cycle elimination procedure. *International Journal of Production Research*, 41(5), 981–994.
- Meng, L., Zhang, C., Shao, X., & Ren, Y. (2019). MILP models for energy-aware flexible job shop scheduling problem. *Journal of Cleaner Production*, 210, 710–723.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., et al. (2013). Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602.
- Narassima, M., Vasudevan, S. K., & Anbuudayasankar, S. (2022). Development of a priority dispatching rule and novel production scheduling algorithm for a job shop. *International Journal of Computer Aided Engineering and Technology*, 16(4), 417–449.
- Pan, Z., Wang, L., Wang, J., & Lu, J. (2021). Deep reinforcement learning based optimization algorithm for permutation flow-shop scheduling. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 1–12.
- Park, J., Chun, J., Kim, S. H., Kim, Y., & Park, J. (2021). Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59(11), 3360–3377.
- Park, I.-B., Huh, J., Kim, J., & Park, J. (2019). A reinforcement learning approach to robust scheduling of semiconductor manufacturing facilities. *IEEE Transactions on Automation Science and Engineering*, 17(3), 1420–1431.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- Pezzella, F., Morganti, G., & Ciaschetti, G. (2008). A genetic algorithm for the flexible job-shop scheduling problem. *Computers & Operations Research*, 35(10), 3202–3212.
- Rooyani, D., & Defersha, F. M. (2019). An efficient two-stage genetic algorithm for flexible job-shop scheduling. *IFAC-PapersOnLine*, 52(13), 2519–2524.
- Saidi-Mehrabad, M., & Fattahi, P. (2007). Flexible job shop scheduling with tabu search algorithms. *The International Journal of Advanced Manufacturing Technology*, 32, 563–570.
- Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust region policy optimization. In *International conference on machine learning* (pp. 1889–1897). PMLR.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815.
- Sobeyko, O., & Mönch, L. (2016). Heuristic approaches for scheduling jobs in large-scale flexible job shops. *Computers & Operations Research*, 68, 97–109.
- Song, W., Chen, X., Li, Q., & Cao, Z. (2022). Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 19(2), 1600–1610.
- Sotskov, Y. N., & Shakhlevich, N. V. (1995). NP-hardness of shop-scheduling problems with three jobs. *Discrete Applied Mathematics*, 59(3), 237–266.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: an introduction*. MIT Press.
- Tang, J., Zhang, G., Lin, B., & Zhang, B. (2011). A hybrid algorithm for flexible job-shop scheduling problem. *Procedia Engineering*, 15, 3678–3683.
- Thenarasu, M., Rameshkumar, K., Rousseau, J., & Anbuudayasankar, S. (2022). Development and analysis of priority decision rules using MCDM approach for a flexible job shop scheduling: A simulation study. *Simulation Modelling Practice and Theory*, 114, Article 102416.
- Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782), 350–354.
- Wang, L., Hu, X., Wang, Y., Xu, S., Ma, S., Yang, K., et al. (2021). Dynamic job-shop scheduling in smart manufacturing using deep reinforcement learning. *Computer Networks*, 190, Article 107969.
- Witty, S., Lee, J. K., Tosch, E., Atrey, A., Clary, K., Littman, M. L., et al. (2021). Measuring and characterizing generalization in deep reinforcement learning. *Applied AI Letters*, 2(4), Article e45.
- Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67–82.
- Xie, J., Gao, L., Peng, K., Li, X., & Li, H. (2019). Review on flexible job shop scheduling. *IET Collaborative Intelligent Manufacturing*, 1(3), 67–77.
- Xing, L.-N., Chen, Y.-W., Wang, P., Zhao, Q.-S., & Xiong, J. (2010). A knowledge-based ant colony optimization for flexible job shop scheduling problems. *Applied Soft Computing*, 10(3), 888–896.
- Yan, Q., Wu, W., & Wang, H. (2022). Deep reinforcement learning for distributed flow shop scheduling with flexible maintenance. *Machines*, 10(3), 210.
- Yuan, E., Cheng, S., Wang, L., Song, S., & Wu, F. (2023). Solving job shop scheduling problems via deep reinforcement learning. *Applied Soft Computing*, 143, Article 110436.
- Yuan, Y., Xu, H., & Yang, J. (2013). A hybrid harmony search algorithm for the flexible job shop scheduling problem. *Applied Soft Computing*, 13(7), 3259–3272.
- Zhang, J.-D., He, Z., Chan, W.-H., & Chow, C.-Y. (2023). DeepMAG: Deep reinforcement learning with multi-agent graphs for flexible job shop scheduling. *Knowledge-Based Systems*, 259, Article 110083.
- Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P. S., & Chi, X. (2020). Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in Neural Information Processing Systems*, 33, 1621–1632.